

Table des matières

Exercices 07 — Traitement distribué avec Spark.....	1
Questions courtes	1
Questions pratiques	2
Diagnostics	2
Questions à développer	3
Mises en situation	3

Exercices 07 — Traitement distribué avec Spark

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur les scripts PySpark que vous venez d'écrire et sur les mesures que vous avez sous les yeux ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 07 et du Labo 07.

Les chiffres à retrouver. Plusieurs énoncés demandent une mesure faite pendant le labo. Si vous avez déjà remis le projet dans l'état du Cours 06 à l'étape 8, refaites la génération de l'étape 1 avant de répondre aux questions pratiques, puis remettez le projet d'aplomb une seconde fois.

Le corrigé est dans `Corrige_07.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les pièces. Nommez le rôle du pilote et celui d'un exécuteur, en une phrase chacun. Dites ensuite ce que les deux deviennent quand on lance Spark en `local[*]`.
2. L'unité de travail. Qu'est-ce qu'une partition, et pourquoi son nombre commande-t-il la durée d'un traitement plus sûrement que le nombre de lignes ?
3. Vingt-sept. Le labo lit 821 fichiers de 145 Ko et obtient 27 partitions de lecture. Expliquez ce nombre à partir des deux réglages qui le produisent.
4. Paresseux. Classez ces appels en transformation ou action : `filter`, `count`, `withColumn`, `write.parquet`, `groupBy`, `show`, `join`, `collect`.
5. Deux fois le travail. Un script fait `df.count()` puis `df.write.parquet(...)` sur le même jeu. Combien de fois les fichiers sources sont-ils lus, et que faudrait-il écrire pour n'en lire qu'une ?
6. Étroit et large. Donnez deux opérations qui ne brassent pas, et trois qui brassent. Dites en une phrase ce que « brasser » veut dire concrètement.

7. Deux cents. D'où vient le nombre 200 dans la ligne `Exchange hashpartitioning(TagName, jour, 200)` du plan, et qu'est-ce qui fait qu'on n'obtient pourtant que trois fichiers en sortie ?
8. Les deux agrégats. Le plan des agrégats montre deux `HashAggregate` autour de l'Exchange. Qu'est-ce que le premier fait, et que se passerait-il s'il n'existait pas ?
9. Le codec. Le même bronze pèse 14 Mo d'un côté et 24 Mo de l'autre, pour les mêmes lignes. Expliquez l'écart, et dites lequel des deux vous garderiez pour un lac conservé cinq ans.
10. Ramener au pilote. `show(20)`, `take(20)`, `limit(20)` et `collect()` demandent tous des lignes. Lequel est dangereux sur un gros jeu, et pourquoi les trois autres ne le sont pas.

Questions pratiques

1. [labo] Les trois qualités. Écrivez le code PySpark qui compte les points du bronze par code de qualité, et dites ce qu'il doit rendre sur l'historien complet.
2. [labo] En zstd. Modifiez l'écriture des agrégats pour qu'elle emploie zstd plutôt que snappy. Donnez la ligne changée et le gain que vous attendez en octets.
3. [labo] Huit partitions. Écrivez la ligne qui ramène le brassage à huit partitions, dites combien de fichiers vous attendez en sortie, et pourquoi ce n'est pas forcément huit.
4. [labo] Sans l'exécution adaptative. Écrivez la ligne qui la désactive, puis dites ce que le dossier de sortie contiendra et ce que cela coûtera en octets et en secondes.
1. [labo] La journée la plus chargée. Écrivez le code PySpark qui donne les trois journées qui portent le plus de points, et le résultat attendu.
2. [labo] Une étiquette, un jour. Écrivez le code qui affiche les dix premiers points de `UTL.CMP2.VIBRATION` du 2025-10-28, dans l'ordre du temps, sans rien ramener d'autre au pilote.
3. [labo] Le contrôle qui tolère. Écrivez la requête DuckDB qui compare les deux jeux d'agrégats en acceptant un écart d'un millième sur la moyenne, et qui rend le nombre de lignes hors tolérance.
4. [labo] Un seul fichier. On vous demande que les agrégats de Spark sortent en un seul fichier, comme ceux du moteur d'une machine. Donnez la façon de le faire, et le prix à payer.

Diagnostics

1. Un collègue mesure Spark ainsi : il chronomètre les lignes `spark.read.csv(...)` et `.filter(...)`, trouve trois millisecondes, et conclut que Spark est plus rapide que DuckDB. Où est l'erreur ?

2. Un script écrit le bronze, puis relit le dossier pour compter, puis écrit un second dérivé à partir du même DataFrame de départ. Il met trois fois plus longtemps que prévu, sans rien faire d'anormal dans les journaux. Que s'est-il passé, et que changez-vous ?
3. Une ingénieure ajoute `.repartition("TagName")` avant son `groupBy` « pour aider Spark à regrouper ». Le résultat est juste et la durée double. Expliquez-lui ce qui se passe, en nommant ce que le plan montrerait.
4. Un contrôle compare les moyennes de DuckDB et de Spark avec `=` et signale six écarts. Un collègue en conclut que Spark a un défaut de calcul. Que répondez-vous, et comment écrivez-vous le contrôle correctement ?
5. Un traitement tourne pendant douze minutes, puis échoue sur `java.lang.OutOfMemoryError` dans le pilote. Le code se termine par `resultat = df.collect()` suivi d'une boucle Python qui écrit un CSV. Corrigez, sans changer ce que le script produit.
6. Un DAG du Cours 06 appelle un script PySpark par `BashOperator`, et la tâche reste bloquée. Dans les journaux d'Airflow, une seule ligne : `docker: command not found`. Où est le problème, et quelles sont les deux façons de le régler ?

Questions à développer

A. Ce cours conclut que Boréal n'a pas besoin de Spark, et fait quand même écrire quatre scripts PySpark. Défendez cette décision pédagogique, puis donnez l'argument le plus fort de celui qui la conteste.

B. Le labo mesure des rapports de 20 à 60 entre les deux moteurs, sur une seule machine. Expliquez ce que cette comparaison prouve et ce qu'elle ne prouve pas, puis dites quelles mesures il faudrait pour trancher honnêtement le jour où le volume aura été multiplié par cent.

C. Un lac de données se dégrade souvent en un dossier de dizaines de milliers de petits fichiers. Expliquez comment un moteur distribué mal réglé y contribue, ce que cela coûte à la lecture, et deux façons de l'éviter dès l'écriture.

Mises en situation

1. La direction de Boréal a lu un article sur le traitement distribué et demande une grappe Spark « pour être prêts ». On vous demande une réponse écrite d'une page. Que mettez-vous dedans, dans quel ordre, et quelle recommandation ?
2. Les Papeteries de Kamouraska exploitent quarante usines instrumentées et gardent huit ans d'historien, soit environ 8 To en Parquet. Elles vous demandent si le moteur d'une machine peut suffire. Quelles questions posez-vous avant de répondre, et quels seuils regardez-vous ?
3. Vous arrivez chez un employeur qui a déjà une plateforme Databricks et trois ingénieurs qui la connaissent. Le volume ne justifie pas le distribué. Recommandez-vous de sortir un traitement de la plateforme pour le mettre sur un moteur d'une machine ? Défendez votre position.
4. Une équipe voisine vous demande de reprendre son traitement PySpark, qui met quarante minutes chaque nuit. Vous n'avez accès qu'au code et à l'interface de Spark. Décrivez votre méthode, dans l'ordre, et les trois premières choses que vous regardez.

5. Un ingénieur propose d'écrire tout le pipeline de Boréal en PySpark, « pour n'avoir qu'un seul outil du bronze au capstone ». Donnez les deux options, ce que chacune coûte en exploitation, et votre recommandation pour cette entreprise-là.